

Menu-Driven Digit Operations in C

Algorithm, Test Cases and C Program

Aim

To write a menu-driven C program using switch-case that accepts a positive integer and performs either of the following operations based on the user's choice: (1) checks whether all digits are unique, or (2) displays the frequency of each digit from 0 to 9.

Algorithm

1. Start.
2. Input a positive integer n.
3. Display the menu: 1. Check whether all digits are unique 2. Find frequency of each digit (0-9).
4. Input the user's choice.
5. Use switch(choice) to select the required operation.
6. For choice 1: initialize freq[10] to 0 and copy n into temp. Extract each digit using temp % 10. If that digit has already occurred, mark the number as Not Unique; otherwise increment its frequency. Continue until temp becomes 0. Display Unique or Not Unique.
7. For choice 2: initialize freq[10] to 0 and copy n into temp. Repeatedly extract digit = temp % 10, increment freq[digit], and divide temp by 10. After processing all digits, display freq_0 through freq_9.
8. For any other menu choice, display Invalid choice.
9. Stop.

Test Case Table

Test Case	Input Number	Choice	Expected Output	Actual Output	Result
TC1	12345	1	Unique	Unique	Pass
TC2	12234	1	Not Unique	Not Unique	Pass
TC3	112333	2	freq_0=0, freq_1=2, freq_2=1, freq_3=3, freq_4=0, freq_5=0, freq_6=0, freq_7=0, freq_8=0, freq_9=0	freq_0=0, freq_1=2, freq_2=1, freq_3=3, freq_4=0, freq_5=0, freq_6=0, freq_7=0, freq_8=0, freq_9=0	Pass
TC4	12345	3	Invalid choice	Invalid choice	Pass

C Program

```
#include <stdio.h>

int main() {
    long long n, temp;
    int choice, digit, freq[10] = {0};
    int unique = 1;

    printf("Enter a positive integer: ");
    scanf("%lld", &n);

    if (n < 0) {
        printf("Please enter a positive integer.\n");
        return 0;
    }

    printf("\nMENU\n");
    printf("1. Check whether all digits are unique\n");
    printf("2. Find frequency of each digit (0-9)\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    temp = n;

    switch (choice) {
        case 1:
            if (temp == 0)
                freq[0] = 1;

            while (temp > 0) {
                digit = temp % 10;
                if (freq[digit] > 0) {
                    unique = 0;
                    break;
                }
                freq[digit]++;
                temp /= 10;
            }

            if (unique)
                printf("Unique\n");
            else
                printf("Not Unique\n");
            break;

        case 2:
            if (temp == 0)
                freq[0] = 1;

            while (temp > 0) {
                digit = temp % 10;
                freq[digit]++;
                temp /= 10;
            }

            for (int i = 0; i < 10; i++)
                printf("freq_%d = %d\n", i, freq[i]);
            break;

        default:
```

```
        printf("Invalid choice\n");  
    }  
  
    return 0;  
}
```

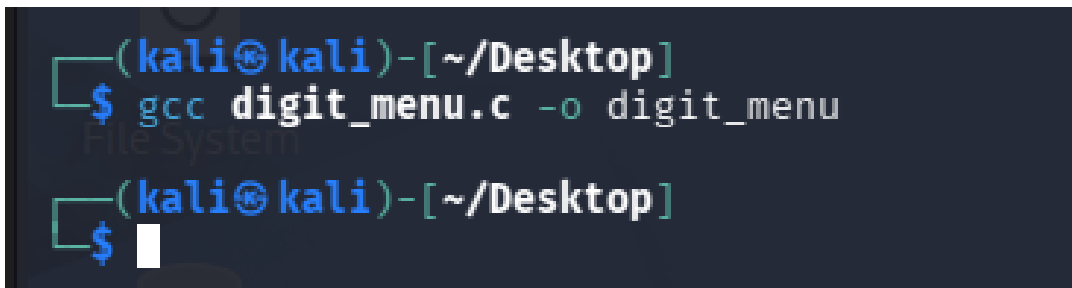
Compilation and Execution

Save the program as digit_menu.c. In the Linux terminal, compile and run it using:

```
gcc digit_menu.c -o digit_menu  
./digit_menu
```

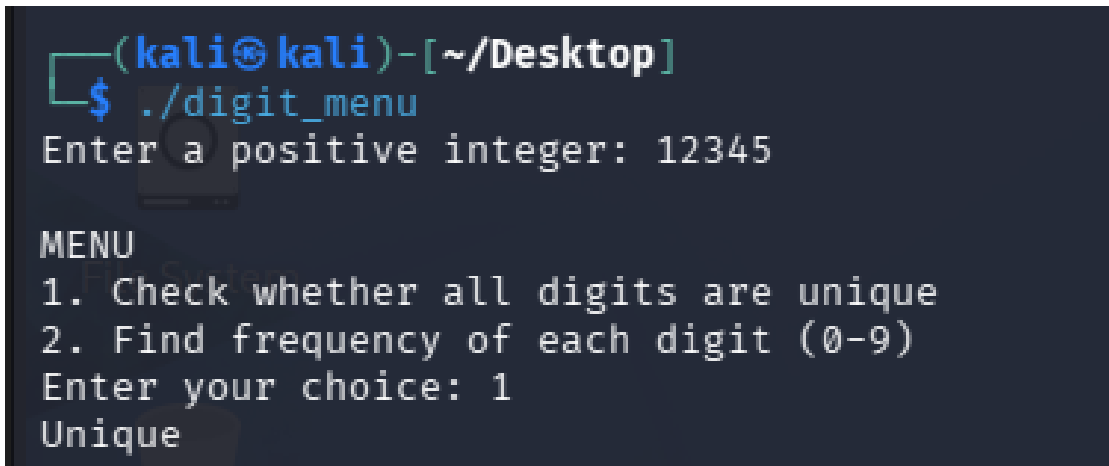
Screenshots / Observations

Screenshot 1: Successful Compilation

A terminal window showing the successful compilation of the program. The prompt is (kali㉿kali)-[~/Desktop]. The user enters \$ gcc digit_menu.c -o digit_menu, and the prompt returns. The user then enters \$, and the prompt returns again.

```
(kali㉿kali)-[~/Desktop]  
$ gcc digit_menu.c -o digit_menu  
  
(kali㉿kali)-[~/Desktop]  
$
```

Screenshot 2: Choice 1 - Unique Case (12345)

A terminal window showing the execution of the program. The prompt is (kali㉿kali)-[~/Desktop]. The user enters \$./digit_menu. The program prompts "Enter a positive integer: 12345". The program then displays a menu with two options: "1. Check whether all digits are unique" and "2. Find frequency of each digit (0-9)". The user enters "1". The program outputs "Unique".

```
(kali㉿kali)-[~/Desktop]  
$ ./digit_menu  
Enter a positive integer: 12345  
  
MENU  
1. Check whether all digits are unique  
2. Find frequency of each digit (0-9)  
Enter your choice: 1  
Unique
```

Screenshot 3: Choice 1 - Not Unique Case (12234)

```
(kali㉿kali)-[~/Desktop]
$ ./digit_menu
Enter a positive integer: 12234

MENU
1. Check whether all digits are unique
2. Find frequency of each digit (0-9)
Enter your choice: 1
Not Unique
```

Screenshot 4: Choice 2 - Digit Frequency Case (112333)

```
(kali㉿kali)-[~/Desktop]
$ ./digit_menu
Enter a positive integer: 112333

MENU
1. Check whether all digits are unique
2. Find frequency of each digit (0-9)
Enter your choice: 2
freq_0 = 0
freq_1 = 2
freq_2 = 1
freq_3 = 3
freq_4 = 0
freq_5 = 0
freq_6 = 0
freq_7 = 0
freq_8 = 0
freq_9 = 0
```

Screenshot 5: Invalid Choice Case (12345, Choice 3)

```
(kali㉿kali)-[~/Desktop]
$ ./digit_menu
Enter a positive integer: 12345

MENU
1. Check whether all digits are unique
2. Find frequency of each digit (0-9)
Enter your choice: 3
Invalid choice
```

Result

The menu-driven C program was written using switch-case, compiled, executed, and tested for checking unique digits and finding digit frequencies. The program produces the expected output for valid test cases and handles an invalid menu choice appropriately.